Endian Views

Document #:	P4030R0 [Latest] [Status]
Date:	2026-02-22
Project:	Programming Language C++
Audience:	SG-9 Ranges SG-16 Unicode LEWG

Reply-to:	Eddie Nolan <eddiejnolan@gmail.com>

Contents

§ 1 Motivation

The main reason for adding these views is to assist users of the UTF transcoding range adaptors (see [P2728R7]). That paper introduces the adaptors `to_utf8`, `to_utf16`, and `to_utf32`, which take as input ranges of `char8_t`, `char16_t`, and `char32_t`. The input and output of these views use native endianness. But users often need to convert to and from UTF encodings with specific endianness: UTF-16LE, UTF-16BE, UTF-32LE, and UTF-32BE.

Rather than introduce a combinatorial explosion of UTF adaptors with various endianness of input and output, we should follow the single responsibility principle and add standard endianness views so users can handle endianness separately.

In addition to UTF transcoding, this facility will help users handle endianness conversions for other streams of data, such as network protocols (TCP/IP, TLS, DNS, etc) and file formats (BMP, TIFF).

§ 2 Before/After Tables

Before	After

```
constexpr vector<uint32_t>
        utf16be_to_utf32be(
    const vector<uint16_t>&
        utf16be_data) {
  return utf16be_data
    | views::transform(
        [](const uint16_t x) {
          if constexpr
        (endian::native ==
        endian::little) {
            return byteswap(x);
          } else {
            return x;
          }
        })
    | views::as_char16_t
    | views::to_utf32
    | views::transform(
        [](const char32_t c) {
          const auto x =
        static_cast<uint32_t>(c);
          if constexpr
        (endian::native ==
        endian::little) {
            return byteswap(x);
          } else {
            return x;
          }
        })
    | ranges::to<vector>();
}
```

```
constexpr vector<uint32_t>
        utf16be_to_utf32be(
    const vector<uint16_t>&
        utf16be_data) {
  return utf16be_data
    | views::from_big_endian
    | views::as_char16_t
    | views::to_utf32
    | views::transform(
        [](const char32_t c) {
          return
        static_cast<uint32_t>
        (c);
        })
    | views::to_big_endian
    | ranges::to<vector>();
}
```

Before	After

```
vector<byte>
        synthesize_tls_client_hello(
    const set<uint16_t>&
        cipher_suites, /* ... */) {
  vector<byte> result;
  // ...
  // TLS ClientHello CipherSuite list
        is a
  // length-prefixed sequence of 16-
        bit
  // big-endian values
  ranges::copy(
    views::concat(
      views::single(
        static_cast<uint16_t>
        (cipher_suites.size())),
```

```
vector<byte>
        synthesize_tls_client_hello(
    const set<uint16_t>&
        cipher_suites, /* ... */) {
  vector<byte> result;
  // ...
  // TLS ClientHello CipherSuite list
        is a
  // length-prefixed sequence of 16-
        bit
  // big-endian values
  ranges::copy(
    views::concat(
      views::single(
        static_cast<uint16_t>
        (cipher_suites.size())),
```

Before

```
      cipher_suites)
    | views::transform(
        [](const uint16_t x) {
          if constexpr (endian::native
        == endian::little) {
            return byteswap(x);
          } else {
            return x;
          }
        })
    | views::transform(
        [](const uint16_t x) {
          return bit_cast<array<byte,
        2>>(x);
        })
    | views::join,
    back_insert_iterator{result});
  // ...
  return result;
}
```

After

```
      cipher_suites)
    | views::to_big_endian
    | views::transform(
        [](const uint16_t x) {
          return bit_cast<array<byte,
        2>>(x);
        })
    | views::join,
    back_insert_iterator{result});
  // ...
  return result;
}
```

§ 3 Dependencies

This paper depends on [P3117R1] “Extending Conditionally Borrowed”.

§ 4 Wording

Add the following subclause to 25.7 [range.adaptors]:

§ 24.7.? Endianness adaptors [range.endianadaptor]

```
struct byteswap-if-native-is-big-endian { // exposition only/
  constexpr auto operator()(auto x) const noexcept {
    if constexpr (endian::native == endian::big) {
      return std::byteswap(x);
    } else {
      return x;
    }
  }
};
```

```
struct byteswap-if-native-is-little-endian { // exposition only/
  constexpr auto operator()(auto x) const noexcept {
    if constexpr (endian::native == endian::little) {
      return std::byteswap(x);
    } else {
      return x;
    }
  }
};
```

If `std::endian::native != std::endian::big` and `std::endian::native != std::endian::little`, the following four range adaptor objects are not provided. Otherwise:

The name `from_little_endian` denotes a range adaptor object ([range.adaptor.object]). Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. If `ranges::range_value_t<T>` does not model `integral`, `from_little_endian(E)` is ill-formed. The expression `from_little_endian(E)` is expression-equivalent to:

— If `T` is a specialization of `empty_view` ([range.empty.view]), then `empty_view<range_value_t<T>>{}`.

— Otherwise, `ranges::transform_view(std::views::all(E), `*`byteswap-if-native-is-big-endian`*`{})`.

The name `from_big_endian` denotes a range adaptor object ([range.adaptor.object]). Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. If `ranges::range_value_t<T>` does not model `integral`, `from_big_endian(E)` is ill-formed. The expression `from_big_endian(E)` is expression-equivalent to:

— If `T` is a specialization of `empty_view` ([range.empty.view]), then `empty_view<range_value_t<T>>{}`.

— Otherwise, `ranges::transform_view(std::views::all(E), `*`byteswap-if-native-is-little-endian`*`{})`.

The name `to_little_endian` denotes a range adaptor object ([range.adaptor.object]). Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. If `ranges::range_value_t<T>` does not model `integral`, `to_little_endian(E)` is ill-formed. The expression `to_little_endian(E)` is expression-equivalent to:

— If `T` is a specialization of `empty_view` ([range.empty.view]), then `empty_view<range_value_t<T>>{}`.

— Otherwise, `ranges::transform_view(std::views::all(E), `*`byteswap-if-native-is-big-endian`*`{})`.

The name `to_big_endian` denotes a range adaptor object ([range.adaptor.object]). Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. If `ranges::range_value_t<T>` does not model `integral`, `to_big_endian(E)` is ill-formed. The expression `to_big_endian(E)` is expression-equivalent to:

— If `T` is a specialization of `empty_view` ([range.empty.view]), then `empty_view<range_value_t<T>>{}`.

— Otherwise, `ranges::transform_view(std::views::all(E), `*`byteswap-if-native-is-little-endian`*`{})`.

§ 4.1 Feature test macro

Add the following macro definition to 17.3.2 [version.syn], header <version> synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_endian_views 20XXXXL // also in <ranges>
```

§ 5 Design Notes

`from_little_endian`/`to_little_endian` and `from_big_endian`/`to_big_endian` do the same thing. We include both names for the sake of pipeline readability; it would be harder to read the pipelines if we used names like `from_or_to_little_endian` or `byteswap_if_native_is_big_endian`.

§ 6 References

[P2728R7] Eddie Nolan. 2024-10-07. Unicode in the Library, Part 1: UTF Transcoding.
https://wg21.link/p2728r7

[P3117R1] Zach Laine, Barry Revzin, Jonathan Müller. 2024-12-15. Extending Conditionally Borrowed.
https://wg21.link/p3117r1